

mem-agent: Equipping LLM Agents with Memory Using RL

Atakan Tekparmak
Dria
atakan@dria.co

Ömer Kaya
Dria
omer@dria.co

October 9, 2025



Code



Model

Abstract

Large language models (LLMs) equipped with tools and a multi-turn action-feedback loop have been a popular direction of research, especially after the recent "Cambrian explosion" of reasoning models following the success of Deepseek-R1. With the increasing capabilities of small, open LLMs, the research in this area has become more accessible to a wider audience, leading to a wide effort in both open and closed research in LLM agents trained with reinforcement learning (RL). One area LLMs fundamentally lack is persistent state over conversations, which could be thought of in the form of a memory. This paper introduces **mem-agent**, a 4B LLM agent trained with GSPO on a scaffold that uses Python tools and markdown files to equip an LLM with memory. We outline a scaffold, data generation pipeline, the training process, and the application of the resulting agent. We also introduce **md-memory-bench**, a hand-crafted benchmark to evaluate LLMs on their proficiency in this scaffold. Using the benchmark, we demonstrate the success of our training setup as our model scores 75% on the benchmark, second only to Qwen3-235B-A22B-Thinking-2507.

1 Introduction

After the introduction of Deepseek-R1 (Guo et al., 2025), the research on LLM agents trained with RL or more specifically, Reinforcement Learning with Verifiable Rewards (RLVR) (Lambert et al., 2024) has become the de-facto standard in post-training research, eclipsing the dominance of SFT (Ouyang et al., 2022) only pipelines in post-training. Since then, many open source frameworks (Brown, 2025; Hu et al., 2024; Sheng et al., 2025; von Werra et al., 2020) have either been released or adopted the techniques used Deepseek-R1, mainly multi-turn GRPO (Shao et al., 2024) with interleaved reasoning. The adoption of GRPO and its derivatives like Dr.GRPO (Liu et al., 2025) and GSPO (Zheng et al., 2025) by both open and closed source labs and researchers set the stage for a "Cambrian explosion" of LLM agents trained with multi-turn RL with interleaved reasoning in the open source research scene.

The biggest use-cases of these LLM agents came, to no surprise, from closed labs and their model and product offerings. State-of-the-art (SOTA) models like GPT-5 (OpenAI, 2025a), Claude 4 Opus & Sonnet (Anthropic, 2025b) are natively trained with multi-turn tool and thinking interleaved RL and have reached wide usage in applications like Claude Code (Anthropic, 2025a), Cursor and open-source options like Zed and Cline.

The overall promise of these LLM agents with multi-turn tool calling, whether they are open or closed, is immense. SOTA models get better and better over time in terms of completing longer and more complex tasks (Kwa et al., 2025), and open models like the Qwen3 series (Yang et al., 2025), GLM-4.5 (Zeng et al., 2025) and Kimi K2 (Team et al., 2025b) have equalled or surpassed the performance of their closed counterparts on many agentic tasks. With all the promise these agents might deliver, an everlasting problem with LLMs and their applications is the problem of context. Context has been an issue to deal with in language modeling long before there were transformer-based LLMs, and it is a problem of three facets:

1. It's a hard limit as LLMs don't work past their set, maximum context length.
2. It's also a moving, soft limit as the effective context length of LLMs are usually significantly lower than the claimed "maximum" context length (An et al., 2024; Li et al., 2024).
3. Longer context requires more memory and compute, which makes all the applications of these agents more expensive.

These problems make it clear that the ideal LLM agent should be context efficient. This becomes even harder when one tries to imbue the agent with extra knowledge with methods like continual pre-training (Ke et al., 2023) or SFT with on-policy samples, as they are gradient-based methods whose compute requirements scale exponentially with increasing context length. A possible remedy for this is Retrieval Augmented Generation (RAG) (Lewis et al., 2020), which sets up a (usually embedding model-based) scaffold for relevant content/document(s) to be retrieved according to the user query, and given to the LLM so it can generate a response accordingly. This method evolved into a sub-field called Agentic RAG (Singh et al., 2025), where the agent is equipped with a scaffold including tools and sometimes graph components. Overall, most of these methods are not dynamic: they give static knowledge to LLMs which cannot be deleted or modified, which can be thought of as equipping them with a read-only database. Ideally, an LLM agent would have a dynamic memory: one that can evolve over time with the usage of the agent, which would be a step forward in the direction of the ever-far promise of "continual learning".

Motivated by the above problems and conclusions we have crafted a scaffold inspired by Obsidian, with Python tools and markdown files in folders. Based on this scaffold, we have determined three major tasks for such an agent to perform:

1. **Retrieval:** Given a user query, the agent should be able to retrieve relevant content from the memory and generate a response accordingly.
2. **Update:** Given a user query, the agent should be able to update the memory with the new information.

3. **Clarification:** Given a user query that lacks clarity/has contradictory information, the agent should be able to ask the user for clarification.

After determining the tasks, we built a graph-based data generation pipeline for the synthetic generation of training samples for the aforementioned tasks. The data generation process was followed by building the training pipeline and improving it over many experiments. Finally, we’ve handcrafted a new benchmark, **md-memory-bench**, to evaluate the performance of our trained agents and other open and closed SOTA models.

Overall, our contributions are fivefold:

1. We’ve crafted a scaffold for any LLM to be able to utilise a markdown-based memory, with a setup similar to dria-agent-a (Tekparmak and andthattoo, 2025b), which itself was inspired by the ReAct framework (Yao et al., 2023),
2. We have engineered a data-generation pipeline for the three important tasks a memory agent should be able to perform: Retrieval, Update and Clarification,
3. We have built and improved upon a training pipeline for the training of **mem-agent**, and gained many insights about the training of such agents in the process, and
4. We curated a new benchmark, **md-memory-bench**, to evaluate the performance of LLMs in the determined tasks.
5. We’ve released **mem-agent-mcp**, a Model Context Protocol (MCP) (Anthropic, 2024) server for any LLM to be able to use **mem-agent** in the background for memory management.

2 Preliminaries

2.1 The State of Memory in LLMs

To understand how we arrived at the scaffold we have in terms design choices, we must first mention the state of research in the sub-field of memory in LLMs. The most recent survey on the topic came out last year (Zhang et al., 2024). The paper goes into great detail about different formats of memory (textual and parametric), memory operations (writing, reading and management), memory sources and applications of LLM agents with memory, for example a Personal Assistant. In it, many important papers in the field are mentioned, like Voyager (Wang et al., 2023), Generative Agents (Park et al., 2023), MemGPT (Packer et al., 2023) and Reflexion (Shinn et al., 2023). The paper, as detailed a survey as it is, does not make a distinction between declarative, procedural and episodic memory, which we deem important when discussing memory in LLM agents.

In Voyager, a Minecraft-playing agent is equipped with a skill library consisting of executable JavaScript code that describes in-game actions that the agent can perform. The model can add new "skills" to this library and modify the ones already present, effectively giving it a procedural memory. In Generative Agents, the agent writes observations to a memory stream which are then retrieved by the model with a deterministic mechanism that takes recency, importance

and relevance into account. This method equips the agent with a semi-episodic memory in a declarative format. MemGPT, which was a primary inspiration for this work, has an intricate scaffold with a queue, a working memory of sorts, and an explicit flow of memory from working memory to long-term memory. In it, the agent is given tools to add data into, modify data in and delete data from the memory system. Finally, in Reflexion, there's a 3 LLM training setup with an Actor, an Evaluator and a Self-Reflector. The Self-Reflector LLM summarizes the experience from a trajectory in a shorter text, which is then added to an "Experience" buffer, which serves as a long-term memory for the Actor LLM.

The argument could be made that the memory systems in Generative Agents and Reflexion are more declarative than they are episodic, as they are not the raw conversation/trajectory data but the LLM generated summaries of that data. For the purposes of this paper, they will be considered declarative and we will deal with a binary classification, with only declarative and procedural memory being considered. This is mainly because true "episodic" memory in LLMs is the trajectory itself, which we don't want to save fully in its raw form due to aforementioned problems with increasing context length. The trajectory can also be considered as the working memory of the LLM, but that is beyond the scope of this paper.

The road to the elusive Artificial General Intelligence (AGI) or for our purposes, an Artificial Generalist Agent (AGA), no doubt crosses through the research and successful application of procedural memory in SOTA agents, LLM or not. The ideal of continual/lifelong learning realistically requires an agent to not only be able to continuously acquire new information, but also be able to acquire new skills, which themselves can be a combination of already acquired skills. The work in this field is even thinner than the work on declarative memory, as crafting training pipelines for procedural memory agents requires an order of magnitude more effort in scaffold design, data/task generation, reward shaping and the overall training setup. AlphaEvolve (Novikov et al., 2025), where a library of programs is continuously "evolved" by an ensemble of LLMs, and DynaSaur (Nguyen et al., 2024), where a library of programs (seeing a pattern here) is continuously updated and used by single LLM, can be attributed to research in procedural memory in LLM agents.

2.2 The State of LLM Agents

LLM agents, more often than not trained with some successor of GRPO and if not, PPO (Schulman et al., 2017), are the main focus of LLM research in 2025, especially in open source research. With improvements both in the frameworks mentioned before and the models themselves, the research on LLM agents has grown in breadth and depth. Following the success of OpenAI's DeepResearch (OpenAI, 2025b), a plethora of papers have been published on "Deep Research" agents (Muennighoff et al., 2025; Xia et al., 2025), which remains a popular task/scaffold in LLM agent research. Beyond deep research, other major application areas for LLM agents are computer use and software engineering, both of which are ever-growing fields with training pipeline after training pipeline, benchmark after benchmark and product after product released in each one (Lin et al., 2025; Liu et al., 2024a; Sager et al., 2025).

LLM agents in their most basic form are LLMs with a list of tool specifications. These agents, given a task/user query:

1. Think about what they will do,
2. Use the given tools each turn by calling them in the appropriate format, and
3. Get feedback from the environment, which often is the tool call results.

Turn after turn until they deem the task completed and reply to the user with a final answer, or until they run out of available conversation turns/tool calls. A lot of open research and products have focused on JSON-based tool calling, which itself was inspired by the first "Function Calling" standard set by OpenAI. In their closed-source offerings like o3, GPT-5 and Codex models however, they opt for Python-based function calling, in which the agent/model responds with a Python code block as an action. The Claude models in their web interface also use JavaScript code blocks for their "analysis" tool, which also suggests that using direct blocks of code for acting is a better choice than providing a JSON with arguments for a single function/tool. This claim is supported by the results of the Dria Pythonic Agent Benchmark (DPAB) (Tekparmak and andthattoo, 2025a), where all but one model evaluated performed better in the same task when using Python code blocks instead of JSON. This is a slightly more effort-taking direction of research as many of the open source LLM RL training frameworks support only JSON-based tool calling by default, and need modification/extension of the codebase to support Python code blocks as actions. Given the fact that the reasoning ability in LLMs are driven by the amount of procedural knowledge in their pre-training data, most of which is in the form of code (Ruis et al., 2024), and the fact that most of that code is probably Python code (GitHub, 2024), and the proven effectiveness of Pythonic tool calling over JSON-based tool calling, we have decided to build our scaffold with Python code as actions.

2.3 The "Low-Hanging Fruit"

With the LLM agent research booming after the advances following GRPO and Deepseek-R1, and the paradigm shift from gradient-based methods to test-time compute methods in the field after the release of OpenAI's o1 (Jaech et al., 2024); the ideas of "learning to learn", "lifelong learning" and "test-time evolution" are pursued more than ever. Studies like AlphaEvolve, GEPA (Agrawal et al., 2025) have further shown the effectiveness of test-time compute while lifelong learning itself is less of a focus. To reach this goal, a reliable memory system is absolutely necessary. Such a system should be:

1. A gradient-free one that uses minimal context and thus, compute,
2. Able to be picked up by current SOTA LLMs, not needing a generation or two of capability increase, and
3. Managed by the agent itself, via tools.

Given that a memory system that abides by these criteria would be used by a ReAct-like agent with multiple turns of thinking, tool calling and feedback from the environment, such an

agent could be trained with RLVR on how well it can use a memory system like that. This, we think, is the "low-hanging fruit" in the field of LLM agents research and we believe research in this direction is a key one on the road to AGI. Because of this, we have decided to design a scaffold that would embody an LLM agent with tools to interact with a dynamic memory system and then create a training pipeline to train an agent with RLVR to learn how to use this scaffold.

3 Scaffold and Task Formulation

3.1 Inspiration

We were mainly inspired by our own usage of Obsidian, a popular note-taking application based on markdown files and folders that contain them. In it, users can link files with others via a special syntax, and then open the "Graph View" of their knowledge base, which has files as nodes and links as edges. The users can also browse the files in a hierarchical filesystem-like view, which provides a 2-dimensional way to organise and update their knowledge base. Because of its many ways to organise data, the human-readability of the markdown format and its ease of use, it's very popular among many users worldwide. For the same reasons, we think a memory system resembling that would be a great fit for an LLM agent to use, because:

1. The widespread usage of markdown makes it already a prime candidate for LLMs, and
2. The hierarchical structure and the traversal via links provide room for the agent to use the memory the best way it can/prefers to

Due to these reasons, we have decided to design our scaffold around an Obsidian-like system managed by Python tools given as blocks of code by the agent.

3.2 Memory Format and Conventions

3.2.1 Layout

The memory lives under a single root directory and follows a structure:

```
memory/
|-- user.md
|-- entities/
    |-- entity_name_1.md
    |-- entity_name_2.md
    |-- ...
```

based on the following file conventions:

- **user.md**: The main file that contains information about the user and their relationships, accompanied by links to the entity file in the format of `[[entities/entity_name.md]]` per relationship.

- `entities/`: The directory that contains the files for the entities.
- `entity_name.md`: The file for the entity with the name `entity_name`. This file follows the same structure as the `user.md` file.

3.2.2 Links

Links are Obsidian-style but must be written exactly, with the full relative path from the memory root and the file extension included:

- `[[entities/dria.md]]` ✓
- `[[dria]]` or `[[entities/dria]]` *not allowed*

3.2.3 File Structure

User and entity files contain sections and subsections declared with `#` and `##`, and key-value pairs with the keys following the `snake_case`. Entity files are also `snake_case`, and are named after the entity. Below are example user and entity files.

`user.md`

```
# User Information
- user_name: Atakan Tekparmak
- birth_date: 2001-09-27
- birth_location: Istanbul, Turkey
- living_location: Groningen, Netherlands
- zodiac_sign: Libra

## User Relationships
- employer: [[entities/dria.md]]
```

`entities/dria.md`

```
# Dria
- industry: AI Infrastructure & Research
- description: Dria provides a universal execution layer and decentralized
                inference network that optimizes and runs AI models across
                heterogeneous hardware.
```

3.3 Tools

We equip the agent with a set of tools that cover the interactions we expect in an Obsidian-like memory: creating, reading, updating and deleting files, navigating folders, and jumping across links.

3.3.1 File Operations

- **create_file(file_path: str, content: str = "") → bool**: Creates a new file at the given path; auto-creates missing parent directories. Returns *True* on success.
- **update_file(file_path: str, old_content: str, new_content: str) → Union[bool, str]**: Replaces occurrences of *old_content* with *new_content* in the target file. Returns *True* on success, or an error message otherwise. This update mechanism was chosen after our initial experiments with a diff-based update mechanism, which proved to be difficult for smaller models to reliably use.
- **read_file(file_path: str) → str**: Reads and returns the contents of the file at *file_path*.
- **delete_file(file_path: str) → bool**: Deletes the file at the given path. Returns *True* on success.
- **check_if_file_exists(file_path: str) → bool**: Returns whether a file exists at *file_path*.

3.3.2 Directory Operations

- **create_dir(dir_path: str) → bool**: Creates a directory (including any missing parents). Returns *True* on success.
- **list_files() → str**: Returns a tree view of the current working directory for a quick, human-readable snapshot of the memory. An example output is:

```
memory/  
|-- user.md  
|-- entities/  
    |-- entity_name_1.md  
    |-- entity_name_2.md
```

- **check_if_dir_exists(dir_path: str) → bool**: Returns whether a directory exists at *dir_path*.

3.3.3 Utilities

- **get_size(file_or_dir_path: str) → int**: Returns the size in bytes of a file or directory; when called with an empty argument, returns the total memory size. This tool is provided to the agent but **not** used in our experiments, as the initially proposed size limits in training were later removed.
- **go_to_link(link_string: str) → str**: Navigates to a linked note via its link string like the one specified in the memory format. Returns the content of the linked note if successful, or an error message otherwise.

3.4 Agent Loop

The main agent loop is as follows:

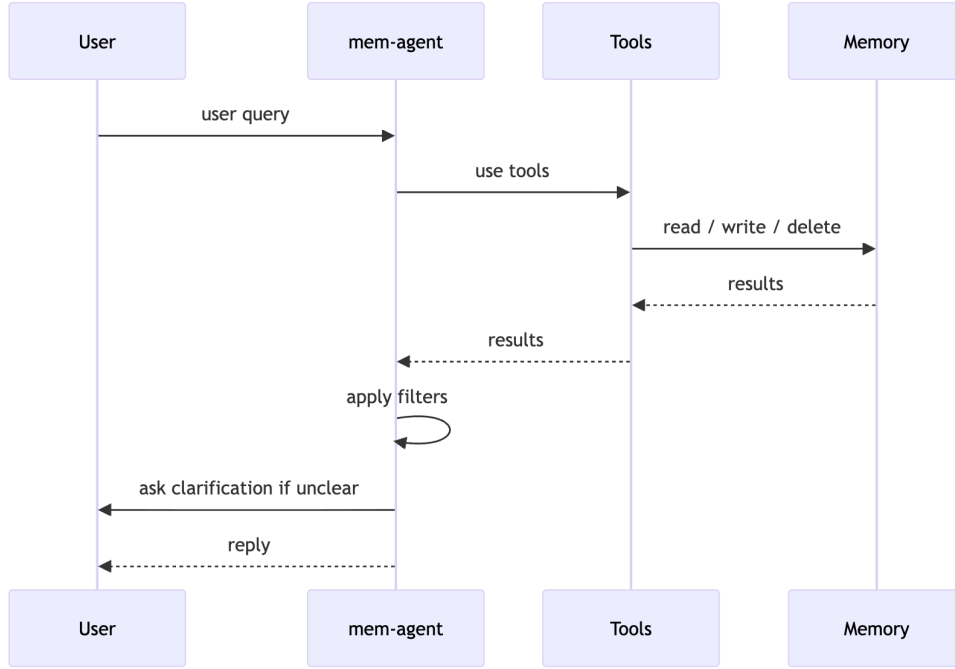


Figure 1: Agent loop diagram

The agent can output content in three different XML blocks:

- **<think>**: The thoughts and planning of the agent w.r.t the user query, the environment feedback if it exists and the relative state towards the completion of the current task. It was originally a **<thoughts>** block but was changed to **<think>** to conform with the format the Qwen3 thinking models were trained on.
- **<python>**: A Python code block that contains the actions the agent wants to take in a given turn. Is fully empty in the turns where the agent replies to the user. The tools given to the agent and the standard library modules are available to the agent.
- **<reply>**: The reply to the user after the agent deems the current task completed. Content within this block is **only** used when the agent wants to reply whereas the **<think>** and **<python>** are mandatory for the agent to provide every turn. In the agent loop, the presence of this block stops the agent interacting with the environment and makes it the user's turn to reply.

The full system prompt for the agent can be found in Appendix .1.

3.5 Task Formulation

Given the memory format, the tools in the scaffold, and the agent loop, we outline three main tasks that an effective agent using this scaffold should be able to perform:

1. **Retrieval:** Given a user query, the agent should be able to retrieve relevant content from the memory and generate a response accordingly. This also contains the subtask of **filtering**, where the agent can receive filters enclosed in `<filter>` and `</filter>` tags and should abide by them accordingly before replying to the user using the retrieved content.
2. **Update:** Given a user query, the agent should be able to update the memory with new information. This includes adding new information or modifying existing information.
3. **Clarification:** Given a user query that lacks clarity/has contradictory information w.r.t the memory, the agent should be able to ask the user for clarification in terms of what to do.

These tasks, we believe, cover the majority of functionality our memory agent should be able to have for continuous, reliable and effective use of the memory scaffold.

4 Data Generation

In this section we will talk about our inspiration for the data generation pipeline given our preliminaries, the do’s and don’ts we learned throughout our iterative development process and the final data generation pipeline we ended up with. Given that the entirety of our training data is synthetically generated, we find the lessons learned during this stage of the study to be highly valuable and relevant to discuss.

4.1 Follow-up to Dria-Agent-a

We reached to two main conclusions after finishing the development and release of dria-agent-a:

1. Python is the way to go as the action format for LLM agents, and
2. Training samples for a lot of different tasks/environments can be synthetically, procedurally generated

The first conclusion helped us with the design of our scaffold. The second conclusion, was the main driver for our data generation pipeline research and development process. Our initial work with dria-agent-a had a pipeline similar to the one in APIGen (Liu et al., 2024b), where the focus was on generating Q&A pairs given a library of functions and prompts, as the focus of our initial agent was to train a decent function-calling agent that proved Python is a better medium than JSON for the task. For this work, because our focus was on instilling the memory management capabilities in the agent, we had to take a different approach to data.

Our initial idea was to implement an entire type system and resolution engine like the one in Prolog (Colmerauer and Roussel, 1993), and add a bunch of primitives and possible constraints

to generate the data for an entire sample with only a set of initial constraints. This was partially inspired by Absolute Zero (Zhao et al., 2025), where the authors give the system a single triplet of input, Python function and output, and then have a proposer propose tasks based on that initial triplet and the solver solve them. The success of that paper proved that an LLM could keep learning not only the given tasks but also overall problem solving and reasoning capabilities with synthetic data.

This idea of generating synthetic data with a resolution engine based on static primitives turned out to be nothing more than a "nerd-snipe" (Munroe, 2007) for the purposes of generating data for this scaffold and task formulation. Not to say that it's a dead-end avenue of research overall, but for the purposes of this study and the potential effort it would take to implement reliably, we opted for a simpler data generation pipeline that would be powered by LLMs and their structured output capabilities.

4.2 What Worked and What Did Not

The base data schema we first implemented was **Persona**, which contained personal information of a person and their relationships with other personas. Using these personas, we generated **Momentary Stories**: logs of what happened in a persona's life with a description and a timestamp. The personas and their momentary stories were then used to generate **Facts** about this persona, in a key-value format wherever applicable. These would then be combined into a **Knowledge Base (KB)** with all the personas and their facts. This knowledge base would then be used to generate multi-turn conversations between the personas, assumed by two different LLMs. The initial goal of generating multi-turn conversations was due to our thinking that a successful small language model (SLM) would need some SFT warmup, as was the norm in post-training research at the time. The momentary stories generated this way were found to be incoherent and inaccurate, which harmed the downstream multi-turn conversation data too. We then got rid of momentary stories and generated the conversations from a KB comprised of only personas and their facts.

We iterated and iterated on this base pipeline, hoping to alleviate the cascading unreliability due to many stages of synthetic data creation without explicit constraints that depend on a single initial seed, which was the "scenario". The data generated, both the KB itself and the multi-turn conversations did not reach the quality we wanted for our training. First, we got rid of the SFT pipeline as we trusted RLVR alone to be enough for training the model we wanted (which it was), and then we started working on a more reliable and deterministic, graph-based pipeline for our data generation.

4.3 Final Data Generation Pipeline

After iterating through various approaches that proved unreliable, we converged on a graph-based pipeline that provides deterministic, high-quality synthetic data generation. The pipeline leverages knowledge graphs as an intermediate representation. Our core insight is to decompose the data generation task into manageable subtasks, each processed by LLMs with focused, limited context. These subtask outputs are then composed at the macro level through a deterministic

graph-based framework that preserves structural integrity and prevents hallucination of multi-hop relationships and attributes.

The pipeline accepts scenario configurations containing world descriptions and generation parameters. Each scenario generates 3–5 user-centric memory instances with 10–30 Q&A pairs per iteration. The critical design choice is using NetworkX directed multigraphs as the intermediate representation, enabling deterministic traversal and modification while supporting multiple edges between nodes, essential for modeling complex relationships like `works_with` and `manages` between the same entities.

4.3.1 Graph Synthesis with Controlled Generation

Graph construction occurs in three deliberately separated phases to prevent cascading hallucinations. First, person and entity stubs are generated with minimal attributes (only id and name), constraining the LLMs’ creative scope. We employ Pydantic models with strict type validation, rejecting any response that includes extraneous fields, a common LLM tendency that corrupts downstream processing.

The relationship inference phase presented unique challenges. LLMs frequently generated edges referencing non-existent nodes or used names instead of IDs. Our solution involves a fallback mechanism: when an edge references an invalid ID, the system attempts name-based resolution before rejection.

Attribute enrichment leverages the graph’s neighborhood context. For each node, we construct a *human-readable* representation including all 1-hop relationships and neighbor attributes. This localized context prevents the LLM from inventing contradictory attributes while maintaining consistency. For instance, ensuring restaurant employees have food-service-related skills rather than unrelated expertise.

4.3.2 Memory Export and Link Management

The memory export generates a 2-hop markdown knowledge base for each focal person node. The system includes all nodes at exactly distance 2 from the focal node, ensuring these entities are only reachable through intermediate connections rather than direct relationships. By definition, these 2-hop neighbors provide information about the focal person’s extended network that cannot be accessed through direct 1-hop traversal.

Cross-references use Obsidian’s double-bracket syntax, but enforcement proved challenging. LLMs consistently attempted shortcuts like `[[john]]` instead of the required `[[entities/john_smith.md]]`. We address this through post-processing validation that enforces full relative paths and adds missing extensions.

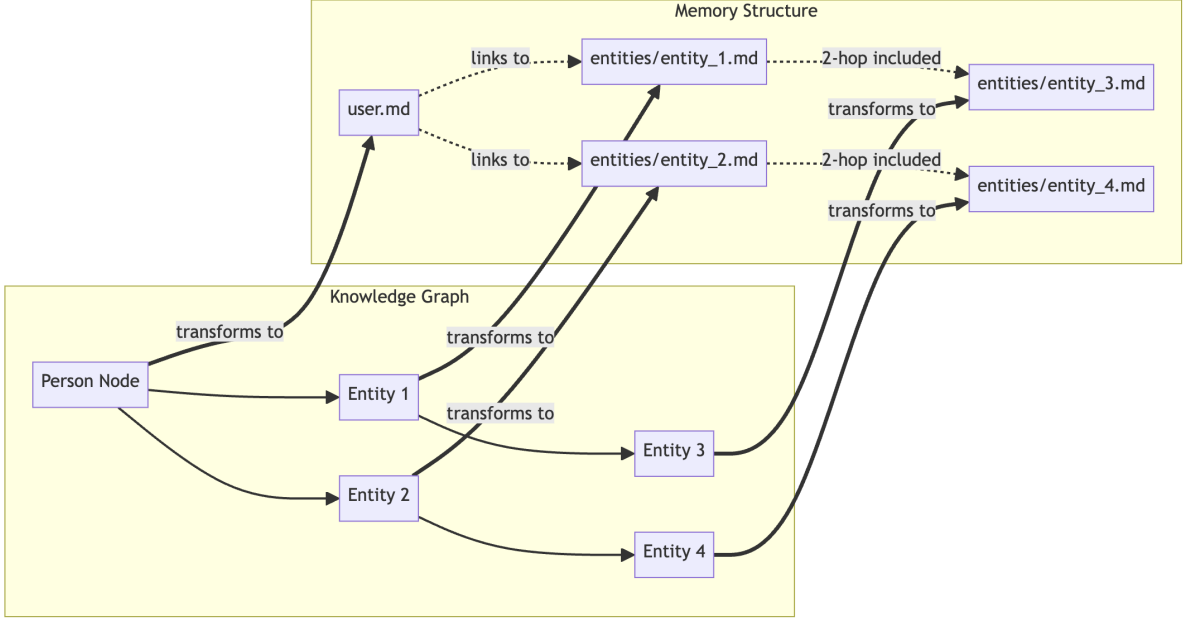


Figure 2: Transformation from knowledge graph to markdown memory structure. Person nodes become `user.md`, while connected entities are transformed into individual markdown files with cross-references. Both 1-hop and 2-hop neighbors are included in the memory structure.

4.3.3 Multi-Hop Question Generation

Generating coherent multi-hop questions requires careful path construction. The system traverses the graph to identify valid paths, then constructs questions that naturally follow relationship chains. A critical issue emerged with relationship directionality: questions must respect edge direction to remain semantically valid. For instance, “What is the age of the person who manages John?” requires an incoming “manages” edge to John, not an outgoing one.

The two-stage generation process prevents a common failure mode where LLMs generate plausible but structurally invalid questions. By constraining initial generation to graph-verified paths, we ensure all questions have corresponding answers in the memory structure.

4.3.4 Update Generation and Diff Computation

Update generation presented the most complex technical challenges. For relationship updates, we must maintain graph consistency when replacing nodes. The system creates a copy of the graph, removes the targeted edge, generates a new node with basic attributes (name and type), and establishes the new connection. While the new node receives minimal initialization, the surrounding context from the world description ensures semantic consistency.

The diff computation required special handling for file creation and deletion. Standard diff libraries assume file existence, but our updates frequently create new entity files. We handle non-existent files by treating them as empty strings in the diff computation, enabling proper diff generation for file creation scenarios.

4.3.5 Validation and Error Recovery

The pipeline implements multi-level validation to catch errors early. JSON Schema validation catches structural issues, while Pydantic models enforce type constraints. Beyond these standard approaches, we implement domain-specific validation including duplicate name detection, which would cause conflicts in the filesystem-based storage model. During graph construction, we attempt name-based node resolution for invalid edge references to recover otherwise valid relationships.

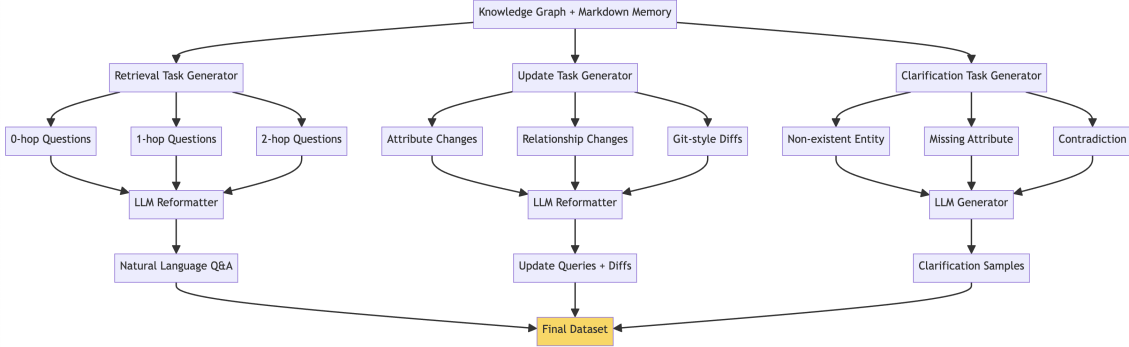


Figure 3: Task-specific data generation flow. The knowledge graph and markdown memory feed into three parallel generators for retrieval, update, and clarification tasks. Each generator produces raw outputs that are reformatted by LLMs into natural language before inclusion in the final dataset.

4.3.6 Auxiliary Generation Techniques

The clarification generator addresses a critical training need: teaching the model to recognize knowledge boundaries. We generate samples by deliberately querying for non-existent entities and attributes, with responses that acknowledge the missing information rather than hallucinating plausible answers. The challenge lies in generating genuinely ambiguous queries. LLMs tend toward overly specific questions that telegraph their invalid nature.

Filter augmentation implements privacy-aware information access. The technical challenge involves generating filters that meaningfully restrict information while maintaining query answerability. We solve this through a three-tier obfuscation model: complete blocking (no information revealed), partial revelation (some attributes visible), and null filters (all information accessible). This distribution ensures the model learns to respect filters without becoming overly restrictive.

This deterministic pipeline achieves reliable, high-quality data generation by carefully managing LLM interactions, implementing robust validation at each stage, and maintaining structural consistency through graph-based orchestration. The key insight is that constraining LLM creativity through limited context and strict validation produces more reliable outputs than attempting to generate complex structures in a single pass.

5 Experiments

In the setup process for our training we have tried a plethora of open-source frameworks to get a minimal Qwen3-4B and GRPO setup on a 8xH100 node working. We tried verifiers, Nemo-RL (NVIDIA, 2025), SkyRL (Cao et al., 2025) and eventually settled on OpenRLHF. This was due to various reasons:

- In both times we tried using **verifiers**, either the Qwen3 models were not properly supported yet or after being able to start the training, we received token id errors. We later found out some of these errors might be caused by the tokenizer vocabulary size mismatch between Hugging Face transformers and vLLM versions of the Qwen3 models. Our attempts to get other open models like Mistral-7B-v3 and Gemma-3-12B (Team et al., 2025a) also failed. We also encountered a lot of NVCC errors due to a lack of clear guidance in GPU and config setup. At the time, the API was immature and slightly over-engineered but has been constantly updated and improved upon by the maintainers. It has since gained widespread adoption, especially after the release of Prime Intellect’s Environments Hub (Intellect, 2025).
- Nemo-RL also suffered from similar problems. Less on the GPU setup/config part but more on the model support and error handling part. We encountered similar token id out of vocabulary issues not only in Qwen3 but other models we tried as well. The API (at least in the time of our trials) was a bit too "OOP-brained" (detectivemittens, 2016) but SOTA SWE agents at the time could write features and identify issues with not much difficulty.
- SkyRL too had an over-complicated API and entrusted the user to deal with a part of the ray (Moritz et al., 2018) setup, which we think is missing the point of a framework. It however had an intuitive `step()` API, which was familiar due to its similarities to the OpenAI Gym’s (Brockman et al., 2016) API which dominated the field of RL for years.

OpenRLHF dealt with all the problems we had with other frameworks: It supported the Qwen3 series, the API was super simple (literally only needed to implement a single `step()` function when we first tried the framework) and the multi-GPU setup was a breeze to figure out compared to the rest. After we managed to get the desired Qwen3 + GRPO on a 8xH100 node working, we were able to properly start our experiments.

5.1 Initial Experiments with Qwen3

In our initial experiments, we had only retrieval (without filters) and update data in the training pipeline and dataset. The retrieval data format and the reward mechanism did not go through any changes throughout our experiments until we added the filter task to the training. The retrieval judge: given a user query, the ground truth and the agent reply, responds with a JSON in the following format:

```
{
  "reply": The exact text of the agent's reply,
  "ground_truth": The exact ground truth fact being checked for,
  "reasoning": Agent's reasoning for its reply,
  "ground_truth_in_reply": true or false
}
```

The agent is instructed to follow the following guideline from our retrieval judge prompt:

- If a `<filter>` block is provided below, you MUST enforce it strictly. If the reply violates the filter (e.g., reveals disallowed info), consider that as not satisfying the ground truth and set `ground_truth_in_reply` to false.

See the full retrieval judge prompt in Appendix .2.

The update task we initially thought of, generated data for and then trained our models on was drastically different than the final one in the "right formula" we found for this study. The initial `write_to_file()` tool required a git-style diff as one of the arguments and consequently the judge was given all the Python blocks used by the agent in concatenated form and a target ground truth diff to look for in those Python blocks. We were seeing no improvements in update performance in the trained model and whatever gains we were making in the training were in retrieval. The initial update judge prompt can be seen in Appendix .3.

We started with Qwen3-4B, then switched to Qwen3-8B for about 20 runs where we played around with hyperparameters, training algorithms and reward shaping. First off, with conflicting results and opinions on whether to have format rewards or not, we opted to go for a format reward as we found the models we trained on had better reward curves than the ones trained without it. Our initial format reward consisted of rewarding the existence of `<python>` blocks and `<think>` blocks. It came back to bite us later, which we will discuss later in a dedicated reward shaping lessons section. We also added a specific potential reward for the first turn, in which the agent would get an extra reward if it checked the existence of or read the `user.md` file. This worked very well and enforced the "check first act second" behaviour we wanted the agent to have.

After the first few experiments, we noticed that the training would degrade around 20 steps in, totally collapse (incoherent model responses and 0 reward) and then the training would crash with the "Token id `<token id>` is out of vocabulary" error (AlexPiche, 2025). This issue is present in both Qwen2.5 and Qwen3 series of models when generating with vLLM. We've tried many solutions to bypass the issue and in the end implemented the workaround suggested in the README of the s1 repository (simplescaling, 2025). This allowed us to continue with our experimentation without crashes, but the model was still degrading and collapsing after some steps. We identified a positive correlation with the length of the `<think>` blocks in the agent's responses and the reward, so we implemented a minimum length for the contents inside and made that a hyperparameter also. That didn't work either, so we started playing around with many variables at the same time. Around this time, the score (validation set reward) and reward (training reward) curves started to diverge, and we switched to using Reinforce Leave One Out (RLOO) (Ahmadian et al., 2024) as our training algorithm.

We modified hyperparameters like the initial Kullback-Leibler (KL) coefficient, KL target, KL horizon, critic and actor learning rates, number of epochs and number of episodes. We also employed epsilon clipping in this stage, with none of our modifications to the training pipeline yielding any success. It seemed like at one point, our training was making the model worse every step. At the time, it also became apparent to the open source community that RL with Qwen3 models was tricky, as it was expected for the pipeline to remove all the `<think>` blocks in the agent’s responses except the last one (@vivekkalyansk, 2025). This, followed by the fact that Qwen2.5 models are exceptionally performant in RLVR settings (Shao et al., 2025), led us to switch to Qwen2.5-Coder (Hui et al., 2024) models.

5.2 Qwen2.5-Coder Experiments

With the switch to Qwen2.5-Coder, specifically the 7B model, we also changed our RL algorithm to Dr.GRPO. After our first run with this model we saw the score and reward curves align again, which was an improvement. What’s more important though, is we discovered a particular behaviour from the model after looking at sample trajectories from each global step: the model was optimizing to get the maximum possible reward by adhering to the format for the maximum allowed turns, 8 turns at the time. The total amount of rewards over 8 steps the model could earn by just following the format was higher than getting the task right in the fifth turn for instance. Reward hacking is studied very well and often in the field of RL, and is considered to be almost inevitable in the first few iterations of an RL environment. To mitigate this, we first tried a crude minus reward if the agent reached maximum turns. The results were immediate, as before the model started the degrade & collapse phase after only 12 steps whereas with a max turns penalty, the scores kept going up way past 20 steps.

We also tried the Qwen2.5-Coder-14B model, and played around with many hyperparameters again, which yielded little to no improvement. The reward and score curves would plateau not much higher than the initial reward and score from the first step. This prompted us to reassess the data and the reward mechanisms for the implemented tasks of retrieval and update. We then decided to completely rewamp the update reward mechanism, where instead of `<python>` blocks and a target diff to be given to the update judge we give it a ground truth and the dump of the agent’s memory folder before and after the trajectory. This took some time to implement with async training and async file operations on the same memories by different agents, but in terms of the data it needed no effort. Our data already contained all we needed which was a ground truth answer. The final update judge prompt can be seen in Appendix .4.

5.3 Reward Shaping Lessons

After we found out the models were abusing the format rewards by using the maximum amount of turns, we had to tabulate the possible rewards for each turn in different scenarios the agent could be in. After an iterative approach to improving, we implemented decreasing rewards for each turn until half of the allowed turns, then increasing negative rewards from half until the maximum turns. We had to meticulously tweak each format reward and success reward and tabulate all possible scenarios afterward, so we could be sure that successfully solving the task, as intended, with the least amount of tool call turns would be preferred by the agent over fishing

for maximum format rewards. We also removed the reward for the `<think>` blocks entirely. Finally, we doubled the reward for successful task completion.

We recommend every researcher in the field, if they are not already doing so, to employ similar methods to be aware of all possible scenarios for rewards in all stages of a trajectory. This helps greatly in identifying possible avenues for reward hacking and in the effort of encouraging behaviours you want the agent to have. The recipe for a good LLM RL setup is three-pronged:

1. Good data that covers the range of behaviours desired in the agent,
2. Good set of hyper-parameters which make use of scale as much as possible within the compute budget, and
3. Fool-proof reward functions and mechanisms that incentivise the model to elicit desired behaviour and not abuse the system.

Most of time all of these are improved iteratively during the experimentation process, which also uses compute. To save compute, time, and effort a preliminary investigation of the reward mechanism and all possible scenarios is essential in finding the right reward functions. In terms of tabulating per-turn cumulative rewards for different scenarios, we've found SOTA proprietary LLMs used inside Cursor to be very capable and reliable.

5.4 Finding the Right Formula

After fixing and shaping the reward mechanisms, we opted for another switch in the algorithm-model pair. We did run with Qwen3-4B-Thinking-2507 and GSPO, where we saw a healthy increasing reward curve until around 15 steps. This looked like one of our best runs so far, especially after evaluating a few selected checkpoints. With this, and us having some free VRAM with the current setup, we decided to increase the batch size and did another run. This run was a "bitter lesson" (Sutton, 2019) for us, as we saw the effects of the scale firsthand. The reward curve did not collapse, even after 120 steps at which point we manually stopped the run. We ran evaluations on selected checkpoints and we saw major improvements in retrieval and update tasks, with varying clarification performance across checkpoints.

As problems to solve, the retrieval and update tasks were practically solved. We then worked on generating the data for clarification, which we talked about in the data generation section, and the reward mechanism for the task. The clarification judge we came up with is very simple and foolproof: Given a user query and an agent reply, the clarification judge has to check if the agent reply asks for a clarification or not. The prompt can be seen in Appendix .5.

After getting clarification setup done, we only had to do two more runs to have a model whose evaluation performance we were satisfied with. In-between these two runs we:

1. Removed sample repetition,
2. Reduced learning rates for both actor and critic,
3. Increased all possible batch sizes,
4. Increased KL target

which all contributed positively to the training and evaluation performance of the model. Our final set of hyperparameters and flags can be found in the `config.json` and `train_agent.sh` files in our repository.

6 Evaluation

Given that the entirety of our training data was synthetically generated we decided to handcraft a benchmark of a few samples for each of our tasks. We thought of evaluation samples for both the "personal assistant" agent application domain we thought of for the training data but also real-life scenarios like customer service and technical support. For the purposes of our evaluation, we considered retrieval with filtering as its own task, **filter**. In this section we will discuss our benchmark, **md-memory-bench**, which in total is comprised of 56 samples. The breakdown of the samples per task is as follows:

- 22 Retrieval, (39.29% of total samples)
- 12 Filter, (21.43% of total samples)
- 11 Clarification, (19.64% of total samples)
- 11 Update, (19.64% of total samples)

6.1 Data Format

All samples, except samples for the **update** task in the benchmark follow the same, simple JSON format. The format is as follows:

```
{
  "question": The user query given to the agent,
  "answer": The "ground truth" referenced by the judge,
  "judge": The per-sample guidelines/remarks given to the judge to assist in
  its reasoning,
}
```

The answer and judge fields in effect indicate the task type to the judge, and because of this data format-based versatility we can use a single system prompt for all tasks except **update**. The format for the **update** task is slightly more complex:

```
{
  "update": The query given to the update agent,
  "judge": The guideline(s) and remark(s) given to the judge,
  "original": Original value for the information to be updated,
  "diff": The new value for the information to be updated,
  "question": The question given to the retrieval agent,
  "answer": The ground truth answer given to the judge,
}
```

The detailed procedure for both update and non-update tasks, and the designations of **update agent** and **retrieval agent** are explained in the following subsection. The full system prompt can be seen in Appendix .6.

6.2 Evaluation Methodology

When a model is evaluated, it iterates over tasks and inside them, over samples sequentially. In the final JSON report generated; the overall score out of 1, per-task scores out of 1 and per-sample metadata like agent reply and judge reasoning are included.

6.2.1 Non-Update Tasks

The basic sequence of a model’s evaluation on a sample, in all tasks except the **update** task, is as follows:

1. The model is given the **question**,
2. The model interacts with the memory and ends the trajectory with a **<reply>**,
3. The judge is given the **question**, **answer** and **reply**,
4. The judge generates a reasoning and a judgement (**CORRECT** or **INCORRECT**), which gives a score of 1.0 and 0.0 respectively.

6.2.2 Update Task

The evaluation procedure for the **update** task is as follows:

1. The **update agent** is given the **update** query,
2. The **update agent** interacts with the memory and ends the trajectory with a **<reply>**,
3. The **retrieval agent** is given the **question**,
4. The **retrieval agent** interacts with the memory and ends the trajectory with a **<reply>**,
5. The judge is given the **question**, **reply** and **judge**,
6. The judge generates a reasoning and a judgement (**CORRECT** or **INCORRECT**), which gives a score of 1.0 and 0.0 respectively.

This double trajectory evaluation process not only measures the update capabilities of the evaluated model but also its retrieval capabilities, as both the update agent and the retrieval agent are initialised from the same model. The implementation of our evaluation can be found in the **evaluation/** directory in our repository. This also applies to our evaluation data, which can be found at **data/eval/**.

7 Results

Including our trained model, we evaluated a total of 11 models on **md-memory-bench**. The results are shown in table 1, where models marked with a * were evaluated on a H100 SXM machine using vLLM and the rest were evaluated using OpenRouter’s API.

Table 1: Performance of tested models on **md-memory-bench**.

Model	Retrieval	Update	Clarification	Filter	Overall
Qwen3-235B-Thinking	0.91	0.64	0.45	1.00	0.79
mem-agent*	0.86	0.73	0.36	0.92	0.75
GLM-4.5	0.77	0.82	0.36	0.92	0.73
DeepSeek-Chat-v3.1	0.68	0.55	0.55	0.83	0.66
Gemini-2.5-Pro	0.73	0.45	0.27	1.00	0.64
Gemini-2.5-Flash	0.77	0.36	0.27	0.92	0.63
GPT-5	0.68	0.55	0.27	0.92	0.63
Claude-Opus-4.1	0.68	0.00	0.82	0.58	0.55
Qwen3-4B-Thinking*	0.45	0.00	0.27	0.75	0.39
Kimi-K2	0.32	0.27	0.18	0.67	0.36

8 Discussion

The Qwen3-235B-A22B-Thinking model leads the pack with an overall score of 0.79 and also in the retrieval tasks alone with a score of 0.91. It also shares the top spot for the filter task alone with Gemini 2.5 Pro, both with a perfect score of 1.00. The update task is done best by GLM-4.5 with a score of 0.82 and the clarification task by Claude Opus 4.1 with the same score. Kimi-K2 is the worst performer overall, but is also the only "non-thinking" model we evaluated.

Our model, **mem-agent**, is the second best performer overall with a score of 0.75. The overall score, and the per-task scores all see a drastic improvement over the base Qwen3-4B-Thinking model. The overall score improves from 0.39 to 0.75, the retrieval score improves from 0.45 to 0.86, filter score improves from 0.75 to 0.92 and the clarification score improves from 0.27 to 0.36. The update task is where we see the biggest improvement, with the score improving from nothing to 0.73. The improvements overall and per task category suggest that our task formulation, data generation and training pipeline were designed and set up correctly.

Given the scores of the tested models on the benchmark, the performance of these models on other benchmarks and their known, general capabilities, we can outline a few main points of discussion:

- The Qwen3-235B-A22B-Thinking being the best performer might be helped by the fact that we optimised the system prompt for the agent on a Qwen3-4B model originally, which would help the biggest model by size in the Qwen3 family, due to them going through similar pre and pos-training pipelines in terms of data and implementation.

- The retrieval task, which was thought of as the first use of the model and the first task we formulated, correlates well on average with both the overall score but also the overall agentic capabilities of the models beyond this benchmark.
- The update task also follows a similar pattern with the big exception of Claude Opus 4.1, which gets none of the update samples correct. We investigated the results one by one, but found no implementation issues on our part but mistakes on the model’s part. The full generated report for the evaluation done on Claude Opus 4.1 can be found `reports/overall/evals/results/claude-opus-4.1.json` in our repository.
- The clarification task is where the "big model smell" (Mowshowitz, 2025) is most apparent, as the model that’s commonly thought as having it, Claude Opus 4.1, is the best performer by a wide margin, followed by Deepseek-v3.1 and Qwen3-235B-A22B-Thinking.
- The filter task is performed well by most models, with again, Claude Opus 4.1 being the exception and surprise in terms of how bad it performs.
- The overall agentic capabilities of tested models correlate with their overall scores on this benchmark, with only Claude Opus 4.1 being the exception.

9 Application

Our resulting model has 4 billion parameters, which in bf16 takes around 8GB of memory. For 8-bit precision it’s 4GB and for 4-bit precision it’s 2GB. This small memory footprint enables for this model to be used in multi-agent applications as well as standalone use. Driven by this, we built a MCP server around our model and released the code in our repository **mem-agent-mcp**. The repository is available [here](#).

The main logic of the MCP server is simple: there’s a single method available for use `use_memory_agent(question: str) -> str` that takes the user query and returns the reply of **mem-agent** after it finishes its trajectory. The only responsibility of the main model used by the user is to determine whether or not to delegate the task to the background memory agent or not. In our testing experience, even small models like Qwen3-4B-Thinking can use this MCP server with great reliability and SOTA proprietary models like Claude Opus 4.1 use it seamlessly. In terms of user experience, different precisions of our model from bf16 to 4-bit perform similarly, with the precision having the most effect on the clarification performance of the model. The 4-bit and 8-bit models tend to hallucinate facts sometimes instead of asking the user for clarification.

In the repository for the MCP server, there’s also a script `chat_cli.py` that allows the user to interact directly with **mem-agent** through a command-line interface. This interface provides the tool interaction turns that the agent takes as well as the final response of the agent, which is useful when observing the model’s behaviour and identifying pitfalls. Due to the configuration setup provided in the repository, both the MCP server usage and the standalone CLI usage interact with the same memory so that the user can choose what mode of interaction they prefer.

10 Conclusion, Limitations and Future Work

In this work, we share excerpts, designs and lessons learned throughout the iterative development process that led to **mem-agent**. Our training data is fully synthetically generated following many works in post-training research that employ a synthetic data generation pipeline for training LLM agents, and also only generated around RLVR without any SFT warmup data following the findings in the Deepseek-R1 paper. The data generation pipeline, much like the training pipeline, was updated and improved over many training runs and their results. These results gave us many insights about the end-to-end pipeline of training an LLM agent with RLVR in a custom environment, and helped us find the "perfect formula" of training Qwen3-4B-Thinking-2507 with GSPO. The benchmark we crafted for this study, **md-memory-bench**, proved to be a reliable way to evaluate the performance of LLM agents on their performance using our scaffold for the tasks we determined. Finally, we released **mem-agent-mcp**, an MCP server for any LLM to be able to use **mem-agent** when needed for managing the memory of the user.

The potential limitations of our work include (but are not limited to):

- The task formulation and data generation were designed around the "personal assistant" task, which might not cover all there is to cover for an LLM agent with declarative memory. This was mainly driven by the fact that this domain + task combination was relatively simple to set up and generate data for, which made it easier for us to test our assumptions about the scaffold and our preliminaries.
- The model we trained is relatively small, and a bigger model could have both better training performance and benchmark results. This is not something that hindered our research process however, as the Qwen3-4B-Thinking-2507 is quite a capable model on its own. Important to note though, the size of this model and the other ones we tried probably affected the reward shaping procedure in terms of how careful we had to be. The small size of the model is also what inspired the MCP use-case, which would otherwise not be possible (but also maybe not necessary) with a bigger model.
- Most likely there are better judging & rewarding mechanisms for the update task in both training and evaluation. We settled on the final training update setup after we were satisfied with the model's performance in the benchmark and our impression after testing it ourselves.
- We didn't generate data, design the scaffold for and train a multi-modal model which would theoretically elicit more complex memory interaction behaviour due to stimuli from different modalities. We opted for a text-only model for the sake of simplicity.

For future research, we would like to explore the following directions:

- **Procedural memory:** The scaffold and memory format in this work focuses on a combination of declarative and a little episodic memory. Moving forward, we plan to explore the end-to-end pipeline (designing the scaffold, data generation, training pipeline and evaluation) for an agent that is given procedural memory.

- **Multi-modal models:** We trained a text-only model for this study, but aim to explore the use of multi-modal models for the next iterations of agents we train. Extra modalities could elicit more complex memory interaction behaviour and result in a more "organic" use of the memory.
- **Bigger/smaller models:** The model we trained is relatively capable, but not capable enough to be the sole model working as the daily driver personal assistant of a user. In future work, we will try out bigger models for the standalone agent use cases and maybe try out smaller models for the "agent-as-a-memory-module" use case. For the latter use case, we also aim to explore Quantization-Aware Training (QAT) (Chen et al., 2024) for the memory module to take even less memory and have more throughput.

Appendix

.1 Agent System Prompt

```

1 Memory Agent System Prompt
2
3 You are an LLM agent with a self-managed, Obsidian-like memory system.
4   You interact with memory using Python code blocks.
5
6 ## CRITICAL: Response Format Rules
7
8 **EVERY response MUST follow this EXACT structure:**
9
10 1. **Always start with '<think>'** - Your reasoning about the query
11    and what memory operations are needed
12 2. **Always follow with '<python>'** - Either:
13    - Python code to interact with memory, OR
14    - Empty tags '<python></python>' if no memory interaction needed
15 3. **Only provide '<reply>' if '<python>' is empty** - Your response
16    to the user
17 4. **The '<python></python>' and '<reply></reply>' MUST be separate,
18    they should not be inside one another, they should be separate
19    blocks**
20
21 ### Valid Response Patterns:
22
23 **Pattern 1: When interacting with memory**
24
25 '''
26 <think>
27 [Your reasoning here]
28 </think>
29
30 <python>
31 [Your Python code here]
32 </python>

```



```

27  '''
28
29  **Pattern 2: When NOT interacting with memory**
30  '''
31  <think>
32  [Your reasoning here]
33  </think>
34
35  <python></python>
36
37  <reply>
38  [Your response to the user]
39  </reply>
40  '''
41
42  **CRITICAL: Always close ALL tags! Missing </think>, </python>, or
    </reply> will cause errors!**
43
44  **NEVER:**
45  - Skip the '<think>' block
46  - Provide text outside of these tags
47  - Use '<reply>' when you have Python code in '<python>'
48  - Respond with plain text after receiving '<result>' blocks
49
50  ## After Receiving '<result>' Blocks
51
52  When you receive '<result>' blocks, you MUST:
53  1. Start a new response with '<think>'
54  2. Analyze the results and decide if more memory operations are needed
55  3. Either provide more Python code OR empty '<python></python>' with a
    '<reply>'
56
57  **Understanding Results:**
58  - '{'variable_name': value}' - Your assigned variables and their values
59  - '{}' - Empty dict means NO variables were assigned (you forgot to
    capture return values!)
60  - If you get '{}', your function calls weren't assigned to variables
61
62  ## Memory API
63
64  **CRITICAL: ALWAYS assign function results to variables or they will
    be LOST!**
65  '''python
66  # CORRECT - Results are captured
67  exists = check_if_file_exists("user.md")
68  content = read_file("user.md")
69
70  # WRONG - Results are lost, you get empty {}

```

```

71 check_if_file_exists("user.md")
72 read_file("user.md")
73 '''
74
75 '''python
76 # File Operations
77 create_file(file_path: str, content: str = "") -> bool # Auto-creates
    parent directories
78 update_file(file_path: str, old_content: str, new_content: str) ->
    Union[bool, str] # Returns True or error message
79 read_file(file_path: str) -> str
80 delete_file(file_path: str) -> bool
81 check_if_file_exists(file_path: str) -> bool
82
83 # Directory Operations
84 create_dir(dir_path: str) -> bool
85 list_files() -> str # Shows tree structure of current working
    directory
86 check_if_dir_exists(dir_path: str) -> bool
87
88 # Utilities
89 get_size(file_or_dir_path: str) -> int # Bytes; empty = total memory
    size
90 go_to_link(link_string: str) -> str
91 '''
92
93 ## File Update Examples
94
95 ### Adding to a table:
96 '''python
97 # Find the last row and add new row after it
98 old_content = "| 2024-03-15 | Joined Premium | Active |"
99 new_content = """| 2024-03-15 | Joined Premium | Active |
100 | 2024-03-20 | Added Family | Active |""
101 result = update_file("user.md", old_content, new_content)
102
103 # ALWAYS check the result!
104 if result != True:
105     # Handle the error - result contains the error message
106     print(f"Update failed: {result}")
107
108 Appending a new section:
109
110 # Find the last line of a section and append after it
111 old_content = "- favorite_color: blue"
112 new_content = """- favorite_color: blue
113 - favorite_food: pizza
114 - favorite_movie: Inception""

```

```

115 result = update_file("user.md", old_content, new_content)
116
117 Appending to a list:
118
119 # Add a new item to an existing list
120 old_content = """## Hobbies
121 - reading
122 - hiking"""
123 new_content = """## Hobbies
124 - reading
125 - hiking
126 - photography"""
127 result = update_file("user.md", old_content, new_content)
128
129 Updating a fact:
130
131 old_content = "- user_age: 25"
132 new_content = "- user_age: 26"
133 result = update_file("user.md", old_content, new_content)
134
135 ## Memory Structure
136
137 ### Root Directory
138 - 'user.md': Personal information & attributes about the user, plus
    relationships to other entities
139 - 'entities/': Information about people, places, organizations, etc.
140   - '[entity_name].md': One file per entity
141
142 ### File Conventions
143 - Dates: YYYY-MM-DD format
144 - File names: snake_case, no spaces
145 - All files use .md extension
146 - New sections in files start with ## headers
147 - Facts stored as: '- fact_name: fact_value'
148 - Cross-references: Use '[[entity_name]]' to link between entities
149
150 ### user.md Structure
151 ```markdown
152 # User Information
153 - user_name: [name]
154 - user_age: [age]
155 - [other attributes]
156
157 ## User Relationships
158 - wife: [[entities/jane_doe.md]]
159 - friend: [[entities/john_smith.md]]
160 - employer: [[entities/google.md]]
161

```

```

162 ## Any other relation
163 - name of entity: Explanation of what markdown files stores.
    [[entities/entity.md]]
164
165 ## Tables
166 - user.md can contain tables for structured data
167 ```
168
169 ## Memory Operation Guidelines
170
171 ### When to Save Information
172 - **Personal facts**: Name, age, preferences, important dates
173 - **Relationships**: Family, friends, colleagues, organizations
174 - **Recurring topics**: Interests, projects, goals that come up
    repeatedly
175 - **Context-dependent info**: Location, job, current situation
176
177 ### When NOT to Save
178 - Temporary information (e.g., "what's 2+2?")
179 - General knowledge questions
180 - One-off calculations or lookups
181
182 ### Entity Creation Rules
183 - Create new entity when: First mention of a person/place/organization
    with substantial information
184 - Update existing entity when: New information about known entity
185 - Attributes (age, location, etc.) belong in the entity file, NOT as
    separate entities
186 !! Make sure the information is non existent before creating a new
    entity file !!
187
188 ### Linking New Entities
189 When creating a new entity file, ALWAYS add a link from the most
    relevant existing file (user.md OR another entity):
190
191 **Example 1: Link from user.md**
192 ```python
193 # First: Create the new entity (entities/ dir created automatically)
194 <python>
195 content = """# Acme Corporation
196 - industry: Technology
197 - location: San Francisco, CA
198 """
199 result = create_file("entities/acme_corp.md", content)
200 </python>
201
202 # After result, add link to user.md
203 <python>

```

```

204 old_content = "## User Relationships"
205 new_content = """## User Relationships
206 - **Employer**: Technology company where user works as senior
      engineer. [[entities/acme_corp.md]]"""
207 result = update_file("user.md", old_content, new_content)
208 </python>
209 '''
210
211 **Example 2: Link between entities**
212 '''python
213 # First: Create new entity
214 <python>
215 content = """# John Smith
216 - relationship: Colleague
217 - department: Engineering
218 """
219 result = create_file("entities/john_smith.md", content)
220 </python>
221
222 # After result, link from company entity
223 <python>
224 old_content = "## Employees"
225 new_content = """## Employees
226 - **Senior Engineer**: Works on backend systems team.
      [[entities/john_smith.md]]"""
227 result = update_file("entities/acme_corp.md", old_content, new_content)
228 </python>
229 '''
230
231 Example link descriptions:
232 - **Primary Residence**: Three-bedroom house with home office and
      garden. [[entities/452_willow_creek_dr.md]]
233 - **Project Lead**: Manages the mobile app development team.
      [[entities/sarah_chen.md]]
234 - **Subsidiary**: Wholly-owned AI research division.
      [[entities/acme_ai_labs.md]]
235
236 ## Important Operating Rules
237
238 1. **Initial Check**: On first interaction, ALWAYS check if 'user.md'
      exists and read its contents before any other operations
239 2. **Be Proactive**: Save relevant information without explicit
      requests
240 3. **Be Selective**: Only save crucial, reusable information
241 4. **No Print Statements**: They won't execute in the Python
      environment
242 5. **Valid Python Only**: Ensure syntactically correct code
243 6. **Execution Timeout**: Keep code blocks concise (5-second timeout)

```

```

244 7. **No Duplicates**: Check existing content before adding information
245 8. **CRITICAL - Use Variables**: ALWAYS capture return values for
    inspection
246     '''python
247     # Good - Result will be visible
248     exists = check_if_file_exists("user.md")
249     content = read_file("user.md")
250     result = update_file("user.md", old, new)
251
252     # Bad - Result will be LOST, you'll get empty {}
253     check_if_file_exists("user.md")
254     read_file("user.md")
255     update_file("user.md", old, new)
256     '''
257     **WARNING**: Function calls without assignment return empty {}
    results!
258 9. **Wait for Results**: After submitting Python code, wait for
    '<result>' blocks before proceeding
259 10. **Error Handling**: ALWAYS check return values from file operations
260 '''python
261 # Good - checks the result
262 result = update_file("user.md", old, new)
263 if result != True:
264     # result contains the 'e'rror message
265
266 # Bad - ignores potential failure
267 update_file("user.md", old, new)
268 '''
269 11. **Your '<python>' block MUST compile under 'ast.parse' and yield
    no 'SyntaxError' **
270 12. **One Operation Per Block**: Execute ONE main operation per
    '<python>' block to avoid errors
271 '''python
272 # Good - separate operations
273 <python>
274 exists = check_if_file_exists("user.md")
275 </python>
276 # Wait for result, then:
277 <python>
278 if exists:
279     content = read_file("user.md")
280 </python>
281
282 # Bad - multiple operations can cause issues
283 <python>
284 exists = check_if_file_exists("user.md")
285 content = read_file("user.md")
286 result = update_file("user.md", old, new)

```

```

287 </python>
288 '''
289
290 ## Memory Maintenance
291
292 - Keep user.md as the source of truth for user information
293 - Ensure cross-references between entities are bidirectional when
    relevant
294 - Periodically review entity relationships for consistency
295
296 ## Correct Search Patterns
297
298 - Use 'list_files()' to see the complete directory structure
299 - Start by reading user.md to understand existing relationships. It's
    your starting point.
300 - Hop between markdowns using cross-references to gather context using
    read_file().
301 - Use 'go_to_link()' to navigate to specific websites if needed, but
    only if it adds significant value to the memory.
302
303 ## Filtering
304
305 In the user query, you might receive a fact-retrieval question that
    includes <filter> tags. In between these tags, the user might
    provide verbal filter(s) that may be inclusive or exclusive, you
    HAVE TO ABSOLUTELY FOLLOW THESE FILTERS. These filters provide
    privacy over user information. If there are no filters, just return
    the answer as is.

```

.2 Retrieval Judge System Prompt

```

1 You are a precise fact-checking judge. Your task is to determine
  whether a specific ground truth fact is present in an agent's reply
  to a given question.
2
3 You will be given:
4 1. A question
5 2. An agent's reply to the question
6 3. A ground truth fact to check for
7
8 Your job is to determine if the ground truth sought for in the
  question is explicitly stated or clearly implied in the agent's
  reply. Be strict in your judgment - the fact must be actually
  present, not just possibly inferred.
9
10 Guidelines:

```

```

11 - A fact is considered present if it is directly stated or
    unambiguously implied
12 - Different phrasings of the same fact should be recognized (e.g., "32
    years old" matches "age: 32")
13 - For relationships, match the exact person and relationship type
    (e.g., "partner(Lars Janssen)" requires both the relationship type
    "partner" AND the name "Lars Janssen")
14 - Partial matches do not count as present (e.g., mentioning "Lars"
    without specifying he's a partner)
15 - Do not make assumptions or inferences beyond what is directly stated
16 - Be case-insensitive in your matching
17 - If a <filter> block is provided below, you MUST enforce it strictly.
    If the reply violates the filter (e.g., reveals disallowed info),
    consider that as not satisfying the ground truth and set
    ground_truth_in_reply to false.
18
19 Question: {{question}}
20
21 Agent's Reply: {{reply}}
22
23 Ground Truth Fact: {{ground_truth}}
24
25 Analyze whether the ground truth fact is present in the agent's reply
    and OUTPUT ONLY a JSON object with exactly these fields (no extra
    text):
26 {
27     "reply": "<the exact text of the agent's reply>",
28     "ground_truth": "<the exact ground truth fact you were checking
        for>",
29     "reasoning": "<your concise reasoning>",
30     "ground_truth_in_reply": <true or false>
31 }
32
33 Rules:
34 - Output ONLY the JSON object. Do not add any backticks, code fences,
    labels, or explanations.
35 - Use strictly the four keys above.
36 - Be objective and precise in your determination.

```

.3 First Update Judge System Prompt

```

1 You are an evaluation judge that assesses whether Python code
    correctly implements target git diffs using the write_to_file()
    function.
2
3 ## Your Task
4

```



```

5 You will receive:
6 1. Concatenated Python code blocks (<python> ... </python>) containing
   function calls wrapped in <actions> ... </actions>
7 2. A target diff specification showing desired file changes wrapped in
   <diff> ... </diff>
8
9 You must evaluate if the Python code correctly applies the target
   diffs and output structured results.
10
11 ## Target Diff Format
12
13 The target diff uses this format:
14
15 '''
16 === filename.md ===
17 @@ -start_line,num_lines +start_line,num_lines @@
18   context line (unchanged)
19 -removed line
20 +added line
21   context line (unchanged)
22
23 === new_file.md ===
24 +++ NEW FILE +++
25 [entire file content]
26
27 === deleted_file.md ===
28 --- DELETED ---
29 '''
30
31 ## Function to Evaluate
32
33 The Python code should contain calls to:
34 '''python
35 write_to_file(file_path: str, diff: str) -> bool
36 '''
37
38 This function applies a git-style unified diff to the specified file.
39
40 ## Evaluation Criteria
41
42 A diff is considered "correctly applied" when:
43
44 1. **File Path Match**: The write_to_file() call targets the correct
   file path (exact match or equivalent path)
45 2. **Diff Content Match**: The diff string passed to write_to_file()
   correctly represents the changes shown in the target diff
46 3. **Special Cases**:
   - For new files (marked with "+++ NEW FILE +++"):

```

```

48     - Either create_file() with the content, OR
49     - write_to_file() with a diff that adds all lines (each line
      prefixed with +)
50     - For deleted files (marked with "--- DELETED ---"):
51     - delete_file() call with the correct path
52
53 ## Important Notes
54
55 - Each file in the target diff (separated by === filename ===) counts
      as one target diff
56 - Multiple change hunks within one file still count as just one target
      diff
57 - The diff passed to write_to_file() must be a valid unified diff
      format
58 - File paths may include directory components (e.g.,
      "entities/person.md")
59 - Order of operations doesn't matter as long as all diffs are applied
60 - If a file has multiple write_to_file() calls, only count it as
      correct if the cumulative effect matches the target
61
62 ## Output Format
63
64 You must output a JSON object with exactly these fields:
65 {
66     "num_correct_diffs_applied": <integer>,
67     "num_target_diffs": <integer>
68 }
69
70 Where:
71 - num_correct_diffs_applied: Count of target diffs that were correctly
      implemented
72 - num_target_diffs: Total count of files in the target diff (each ===
      section is one)
73
74 ## Example Analysis
75
76 If the target shows changes to 3 files and the Python code correctly
      implements changes to 2 of them:
77 {
78     "num_correct_diffs_applied": 2,
79     "num_target_diffs": 3
80 }
81
82 Below is the concatenated python blocks and the target diff:
83
84 <actions>
85 {{python_blocks}}
86 </actions>

```

```
87
88 <diff>
89 {{diff}}
90 </diff>
```

.4 Second Update Judge System Prompt

```
1 You are an evaluation judge that assesses whether an agent has
  successfully made the required updates to a memory system based on
  a user's query.
2
3 ## Your Task
4
5 You will receive:
6 1. The original user query requesting updates to be made
7 2. The initial state of the memory folder (before any actions)
8 3. The final state of the memory folder (after all actions)
9
10 You must evaluate if the agent correctly implemented the updates
    requested by the user.
11
12 ## Input Format
13
14 You will receive:
15 - <user_query>: The original request from the user describing what
    updates should be made
16 - <initial_folder_dump>: The complete state of the memory folder
    before any actions
17 - <final_folder_dump>: The complete state of the memory folder after
    all agent actions
18
19 ## Evaluation Criteria
20
21 The updates are considered "correctly applied" when:
22
23 1. **Understanding Intent**: The agent correctly understood what the
    user wanted to update
24 2. **Completeness**: All requested changes have been made
25 3. **Correctness**: The changes match what was requested (e.g.,
    correct values, proper formatting)
26 4. **No Unnecessary Changes**: The agent didn't make changes beyond
    what was requested
27
28 ## Important Considerations
29
30 - Focus on whether the user's intent was fulfilled, not on
    implementation details
```

```

31 - The agent may use different approaches to achieve the same result -
    this is acceptable
32 - New files may be created or existing files modified as needed
33 - The folder structure may change if that serves the user's request
34 - Minor formatting differences are acceptable as long as the content
    is correct
35
36 ## Output Format
37
38 You must output a JSON object with exactly these fields:
39 {
40   "reasoning": "<string explaining your evaluation>",
41   "success": <boolean>
42 }
43
44 Where:
45 - reasoning: A brief explanation of your evaluation decision
46 - success: true if the updates were correctly applied, false otherwise
47
48 ## Example
49
50 If a user asks to "add a new person named John Smith with age 25" and
    the agent creates the appropriate file or entry with this
    information, you would output:
51 {
52   "reasoning": "The agent successfully created an entry for John Smith
    with age 25 as requested",
53   "success": true
54 }
55
56 Below are the user query and folder states:
57
58 <user_query>
59 {{user_query}}
60 </user_query>
61
62 <initial_folder_dump>
63 {{initial_folder_dump}}
64 </initial_folder_dump>
65
66 <final_folder_dump>
67 {{final_folder_dump}}
68 </final_folder_dump>

```

.5 Clarification Judge System Prompt

```

1 You are a precise evaluator of CLARIFICATION responses for a
   memory-augmented agent.
2 Your goal is to judge whether the agent responded by seeking
   clarification rather than fabricating facts.
3
4 Inputs:
5 - Question (from user):
6 {{question}}
7
8 - Agent reply:
9 {{reply}}
10
11 Task definition:
12 The agent should handle three common clarification scenarios without
   hallucinating:
13 1) Non-existing entity: The question references an entity not present
   in memory -> reply should say it cannot find that entity (or lacks
   info) and ask the user to confirm or provide identifying details.
14 2) Non-existing attribute (of an existing entity): The question
   requests an attribute not present -> reply should state the
   attribute is missing and ask for the exact info needed.
15 3) Contradiction: The question contains a fact that conflicts with
   memory -> reply should point out the mismatch and ask the user
   which version is correct.
16
17 General requirements for a good clarification reply:
18 - Do not present an unverified fact as truth.
19 - Ask at least one concrete, helpful follow-up question to resolve the
   ambiguity.
20 - Be concise and natural (1-2 sentences is fine, but longer is
   acceptable if helpful).
21 - If the user included <filter>...</filter> constraints in the
   question, the reply must respect them (no leaking filtered info).
22
23 Output JSON (strict):
24 {
25   "reasoning": string,
26   "success": boolean
27 }

```

.6 Evaluation Judge System Prompt

```

1 You are an agentic task evaluation judge. Your role is to evaluate the
   quality of an assistant model's answer based on a given question,
   the provided answer, and a specific judging preference. The
   assistant model uses markdown files to store the memory of the
   conversation.

```

2

3 Consider the following question that was asked to the assistant:

4 <question>

5 {{question}}

6 </question>

7

8 Here is the CORRECT answer

9 <correct_answer>

10 {{correct_answer}}

11 </correct_answer>

12

13 Here is the answer provided by the assistant:

14 <answer>

15 {{answer}}

16 </answer>

17

18 Your task is to evaluate this answer based on the following judging preference:

19 <judge>

20 {{judge}}

21 </judge>

22

23 Analyze the answer carefully, considering the following aspects:

24 1. Relevance to the question

25 2. Accuracy of information

26 3. Completeness of the response

27 4. Consistency with the provided memory content

28 5. Adherence to the specific judging preference

29

30 ATTENTION:

31 - Correct answer might be empty, in which case the assistant's answer is subjective and should be evaluated based on the judging preference.

32 - Judging preference will emphasize what the assistant should focus on ideally and what kind of information is expected to be used to form an answer that is retrieved from the memory.

33 - Judging preference can be empty, in which case the answer should be evaluated based on the question and answer content alone.

34 - If the <question> includes a <filter>...</filter> block, STRICTLY enforce those constraints. If the assistant reveals information disallowed by the filter, mark the answer as INCORRECT even if other parts are correct. If there are filters, the judging preference and filters take precedence over the correct answer, this is VERY IMPORTANT.

35

36 In your evaluation, provide a detailed reasoning for your judgment. Consider both the strengths and weaknesses of the answer. If there are any discrepancies or issues, point them out clearly.

```

37
38 After your analysis, provide your final judgment on whether the answer
    is correct or not based on the given criteria. Use the following
    format for your response:
39
40 <reasoning>
41 [Provide your detailed reasoning here, addressing each of the aspects
    mentioned above]
42 </reasoning>
43
44 <judgment>
45 [State your final judgment: CORRECT or INCORRECT]
46 </judgment>

```

References

- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gega: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms. *arXiv preprint arXiv:2402.14740*, 2024.
- AlexPiche. Bug: Qwen/qwen2.5-1.5b-instruct generates out of vocabulary tokens, September 2025. URL <https://github.com/vllm-project/vllm/issues/13175>. GitHub issue, vllm-project/vllm.
- Chenxin An, Jun Zhang, Ming Zhong, Lei Li, Shansan Gong, Yao Luo, Jingjing Xu, and Lingpeng Kong. Why does the effective context length of llms fall short? *arXiv preprint arXiv:2410.18745*, 2024.
- Anthropic. Introducing the model context protocol, November 2024. URL <https://www.anthropic.com/news/model-context-protocol>. Accessed: 2025-09-17.
- Anthropic. Claude code, February 2025a. URL <https://www.anthropic.com/news/claude-3-7-sonnet>. Accessed: 2025-09-16.
- Anthropic. System card: Claude opus 4 & claude sonnet 4, May 2025b. URL <https://www.anthropic.com/claude-4-system-card>. Accessed: 2025-09-16.
- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, June 2016. URL <https://arxiv.org/abs/1606.01540>.
- William Brown. Verifiers: Reinforcement learning with llms in verifiable environments, 2025. URL <https://github.com/willccbb/verifiers>.

- Shiyi Cao, Sumanth Hegde, Dacheng Li, Tyler Griggs, Shu Liu, Eric Tang, Jiayi Pan, Xingyao Wang, Akshay Malik, Graham Neubig, Kourosh Hakhmaneshi, Richard Liaw, Philipp Moritz, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Skyrl-v0: Train real-world long-horizon agents via reinforcement learning, 2025.
- Mengzhao Chen, Wenqi Shao, Peng Xu, Jiahao Wang, Peng Gao, Kaipeng Zhang, and Ping Luo. Efficientqat: Efficient quantization-aware training for large language models. *arXiv preprint arXiv:2407.11062*, 2024.
- Alain Colmerauer and Philippe Roussel. The birth of prolog. *ACM SIGPLAN Notices*, 28(3): 37–52, 1993. doi: 10.1145/155360.155362. Historical account of Prolog’s creation.
- detectivemittens. Resources for learning recursion, March 2016. URL https://elixirforum.com/t/resources-for-learning-recursion/210?utm_source=chatgpt.com. elixir forum discussion.
- GitHub. Octoverse: Ai leads python to top language as the number of global developers surges, December 2024. URL <https://github.blog/news-insights/octoverse/octoverse-2024/>. Accessed: 2025-09-16.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Jian Hu, Xibin Wu, Wei Shen, Jason Klein Liu, Zilin Zhu, Weixun Wang, Songlin Jiang, Haoran Wang, Hao Chen, Bin Chen, et al. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143*, 2024.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- Prime Intellect. Environments hub: A community hub to scale rl to open agi, August 2025. URL <https://www.primeintellect.ai/blog/environments>. Blog post.
- Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- Zixuan Ke, Yijia Shao, Haowei Lin, Tatsuya Konishi, Gyuhak Kim, and Bing Liu. Continual pre-training of language models. *arXiv preprint arXiv:2302.03241*, 2023.
- Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, et al. Measuring ai ability to complete long tasks. *arXiv preprint arXiv:2503.14499*, 2025.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhui Chen. Long-context llms struggle with long in-context learning. *arXiv preprint arXiv:2404.02060*, 2024.
- Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. Showui: One vision-language-action model for gui visual agent. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 19498–19508, 2025.
- Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. Large language model-based agents for software engineering: A survey. *arXiv preprint arXiv:2409.02977*, 2024a.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding rl-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh RN, et al. Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets. *Advances in Neural Information Processing Systems*, 37: 54463–54482, 2024b.
- Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. Ray: A distributed framework for emerging {AI} applications. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*, pages 561–577, 2018.
- Zvi Mowshowitz. OpenAI Model Differentiation 101. Don’t Worry About the Vase (Substack), July 2025. URL <https://thezvi.substack.com/p/openai-model-differentiation-101>. Explains OpenAI model names; uses the colloquial “big model smell”.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.
- Randall Munroe. Nerd sniping. xkcd (Comic #356), December 2007. URL <https://xkcd.com/356/>. Introduces the slang “nerd sniping” / “nerd-snipe”.
- Dang Nguyen, Viet Dac Lai, Seunghyun Yoon, Ryan A Rossi, Handong Zhao, Ruiyi Zhang, Puneet Mathur, Nedim Lipka, Yu Wang, Trung Bui, et al. Dynasaur: Large language agents beyond predefined actions. *arXiv preprint arXiv:2411.01747*, 2024.
- Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian,

- et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- NVIDIA. Nemo rl: A scalable and efficient post-training library, June 2025. URL <https://github.com/NVIDIA-NeMo/RL>. GitHub repository.
- OpenAI. Gpt-5 system card, August 2025a. URL <https://openai.com/index/gpt-5-system-card/>. Accessed: 2025-09-16.
- OpenAI. Introducing deep research, February 2025b. URL <https://openai.com/index/introducing-deep-research/>. Accessed: 2025-09-16.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.
- Laura Ruis, Maximilian Mozes, Juhan Bae, Siddhartha Rao Kamalakara, Dwarak Talupuru, Acyr Locatelli, Robert Kirk, Tim Rocktäschel, Edward Grefenstette, and Max Bartolo. Procedural knowledge in pretraining drives reasoning in large language models. *arXiv preprint arXiv:2411.12580*, 2024.
- Pascal J Sager, Benjamin Meyer, Peng Yan, Rebekka von Wartburg-Kottler, Layan Etaiwi, Aref Enayati, Gabriel Nobel, Ahmed Abdulkadir, Benjamin F Grewe, and Thilo Stadelmann. A comprehensive survey of agents for computer use: Foundations, challenges, and future directions. *arXiv preprint arXiv:2501.16150*, 2025.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shaolei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, et al. Spurious rewards: Rethinking training signals in rlvr. *arXiv preprint arXiv:2506.10947*, 2025.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.
- simplescaling. s1: Simple test-time scaling, known issues section in readm, January 2025. URL <https://github.com/simplescaling/s1?tab=readme-ov-file#known-issues>. GitHub issue, simplescaling/s1.
- Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. Agentic retrieval-augmented generation: A survey on agentic rag. *arXiv preprint arXiv:2501.09136*, 2025.
- Rich Sutton. The bitter lesson, March 2019. URL <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>. Online blog.
- Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025a.
- Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025b.
- Atakan Tekparmak and andthattoo. Dria pythonic agent benchmark (dpab). Hugging Face Blog, January 2025a. URL <https://huggingface.co/blog/andthattoo/dpab-a>. Published January 15, 2025. Accessed September 16, 2025.
- Atakan Tekparmak and andthattoo. Python is all you need? introducing dria-agent-a. Hugging Face Blog, January 2025b. URL <https://huggingface.co/blog/andthattoo/dria-agent-a>. Published January 10, 2025. Accessed September 16, 2025.
- @vivekkalyansk, July 2025. URL <https://x.com/vivekkalyansk/status/1948296877180670039>. Reply on X.com.
- Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. Trl: Transformer reinforcement learning, 2020. URL <https://github.com/huggingface/trl>.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Ziyi Xia, Kun Luo, Hongjin Qian, and Zheng Liu. Open data synthesis for deep research. *arXiv preprint arXiv:2509.00375*, 2025.

- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, et al. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models. *arXiv preprint arXiv:2508.06471*, 2025.
- Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents, 2024. URL <https://arxiv.org/abs/2404.13501>, 2024.
- Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.